RAG-Based Customer Support Assistant

Technical Documentation

Name: Devyani S. Bharare

College: P R Pote College Of Engineering And Management, Amravati.

Course: Internship Project

Date: April 2026

📘 1. Introduction

◈ What is RAG?

Retrieval-Augmented Generation (RAG) is a technique that combines:

- **Information Retrieval** (from documents)
- **Text Generation** (using LLM)

👉 It retrieves relevant data first, then generates an answer.

◈ Why RAG is Needed

- Reduces hallucination (wrong answers)
- Uses real company data
- Improves accuracy of responses

◈ Use Case

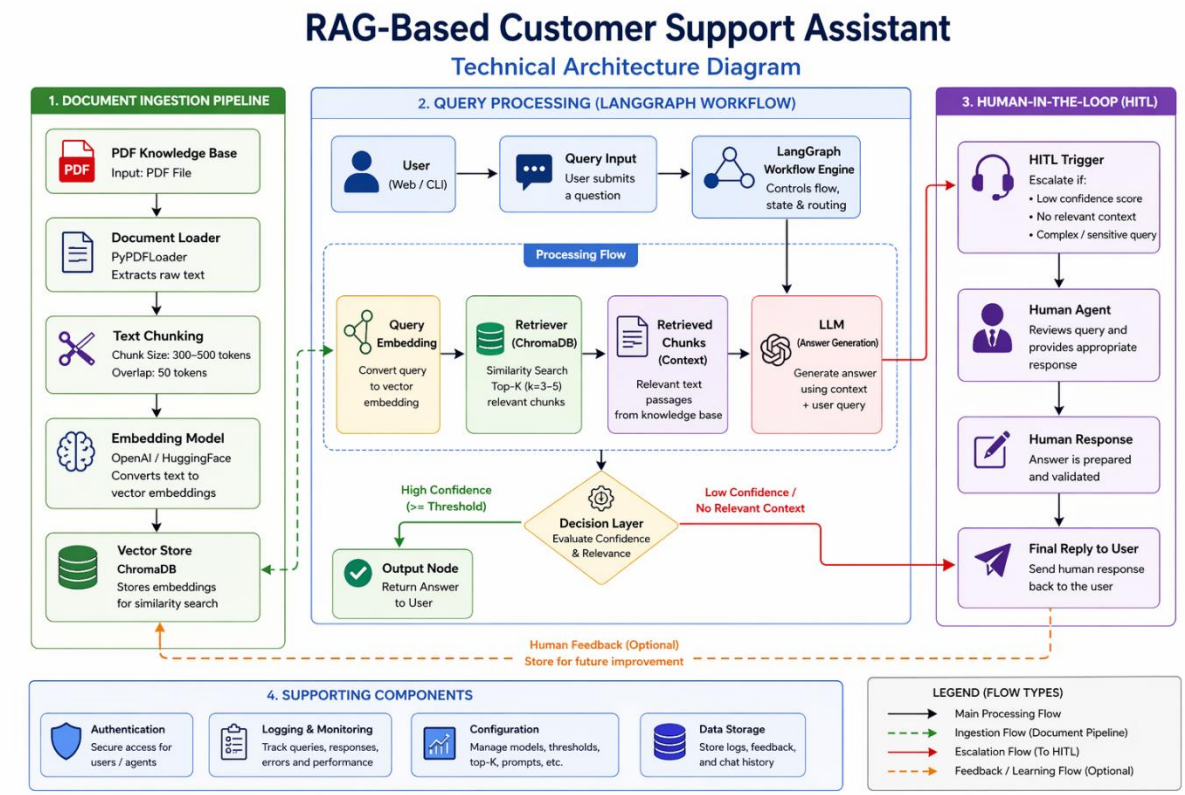
Customer Support Assistant:

- Answers FAQs from PDF
- Handles repetitive queries
- Escalates complex queries to humans

📓 2. System Architecture Explanation

The system consists of:

- **User Interface** → accepts query
- **Document Pipeline** → processes PDF
- **Vector Database (ChromaDB)** → stores embeddings
- **Retriever** → finds relevant chunks
- **LLM** → generates answer
- **LangGraph** → controls workflow
- **HITL Module** → handles escalation



◈ **Working Flow:**

1. User submits query
2. Query is converted into embedding
3. Retriever fetches relevant chunks
4. LLM generates answer
5. Decision layer checks confidence
6. Either:
    - Send answer
    - Escalate to human

📘 3. Design Decisions

◈ **Chunk Size (300–500 tokens)**

- Smaller chunks → faster retrieval
- Larger chunks → better context
  👉 Balanced choice used

◈ **Embedding Strategy**

- Semantic embeddings used
- Captures meaning instead of keywords

◈ **Vector DB (ChromaDB)**

- Lightweight and easy to use
- Fast similarity search

◈ Retrieval Approach

- Top-K retrieval (k = 3–5)
- Improves relevance of answers

◈ Prompt Design

- Context + Query passed to LLM
- Ensures grounded answers

📘 4. Workflow Explanation (LangGraph)

LangGraph is used to control execution.

◈ Nodes:

- Input Node → receives query
- Processing Node → retrieval + generation
- Decision Node → checks confidence
- Output Node → returns response

◈ State Transition:

Query → Retrieved Data → Answer → Confidence → Final Output

◈ Role of LangGraph:

- Maintains state
- Enables conditional routing

- Ensures structured workflow

📘 5. Conditional Logic

Condition	Action
High confidence	Return Answer
Low confidence	Escalate
No Data Found	Escalate

🔹 **Logic Used:**

- Based on similarity score
- Based on answer completeness

📘 6. HITL Implementation

🔹 **Purpose:**

To handle cases where AI is unsure.

🔹 **When Triggered:**

- Low confidence
- Missing context
- Complex queries

🔹 **Flow:**

Query → Human → Response → User

🔹 **Benefits:**

- Improves reliability
- Handles edge cases

◈ **Limitation:**

- Requires human availability

📘 7. Challenges & Trade-offs

Challenge	Trade-Off
Chunk Size	Small = fast, Large = better context
Retrieval accuracy	High accuracy = more computation
Cost	More LLM calls = higher cost

📘 8. Testing Strategy

◈ **Test Cases:**

- FAQ queries
- Unknown queries
- Edge cases

◈ **Example Queries:**

- “What is refund policy?”
- “How to contact support?”
- “Explain something not in PDF”

📘 9. Future Enhancements

- Multi-document support
- Chat memory
- Feedback learning system

- Deployment (AWS / Cloud)